DDoS Attack Detection on MQTT Protocol Using Semi-supervised DBSCAN and Support Vector Machine Model

2023 11th International Conference on Information and Communication Technology (ICoICT) | 979-8-3503-2198-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICoICT58202.2023.10262594

1st Muhammad Ikhsanudin
School of Computing
Telkom University
Bandung, Indonesia
ikhsanudinmuh@student.telkomuniversity.ac.id

2nd Vera Suryani
School of Computing
Telkom University
Bandung, Indonesia
verasuryani@telkomuniversity.ac.id

3rd Rizka Reza Pahlevi
School of Computing
Telkom University
Bandung, Indonesia
rizkarezap@telkomuniversity.ac.id

***Abstract*—Internet of Things (IoT) is a system of connected objects with sensors, software, control systems and protocols. One of the protocols widely used in IoT is Message Queue Telemetry Transport (MQTT). Considering that users of these IoT devices can control their devices from anywhere makes them susceptible to various types of attacks. Distributed Denial of Service (DDoS) is a common attack vector in the IoT. Among the methods that can be applied for identifying this attack is machine learning. In previous research, DDoS detection was accomplished using a singular SVM. The precision and f1-score generated by this solitary SVM are still inadequate. This research combines SVM with other machine learning techniques in an effort to enhance SVM's accuracy and f1-score. In this research, semi-supervised DBSCAN and SVM models were utilized. We use three datasets in this research, namely IoTID20, simulation, and CICDDOS2018. The proposed model has the ability to detect DDoS attacks with a 99.6% accuracy, 99.6% f1-score, and 0.8% false alarm rate, respectively.**

Keywords—Internet of Things, MQTT, Semi-supervised model, DBSCAN, SVM

I. Introduction

Sensors, software, control systems, and protocols are some of the devices that are connected by the object systems, namely Internet of Things (IoT) [1]. MQTT, also known as Message Queue Telemetry Transport, is one of the most used protocols for the Internet of Things [2]. Smart devices are an example of how the IoT is being used in daily life. These devices undoubtedly include the personal information of the user. These IoT devices allow users to operate their devices from anywhere, which makes them vulnerable to various types of cyber attacks [3].

Distributed Denial of Service (DDoS) attacks are one type of frequent attack in IoT environment. This attack happens when numerous clients continually send requests to the server, which can render the server overburdened with requests to process. The IoT device's battery life may be impacted by this attack's potential to turn it on constantly. A distinctive botnet infection known as Mirai has emerged in recent years, it is capable of damaging DDoS attacks that have broken many IoT devices [3].

There are several techniques to identify attacks, employing machine learning is one of the most effective approaches [3]. In previous studies [4], A single SVM has been used to detect DDoS on the SDN control plane. This singular SVM's accuracy and f1-score are still not very good. In this research, SVM is combined with other machine learning methods to increase accuracy and the f1-score. Based on previous research [5], DBSCAN has been tested to detect DDoS attacks on cloud platforms. According to the study, DBSCAN can be combined with other machine learning techniques to increase accuracy. Therefore, we conducted this research to test whether the integration of DBSCAN and SVM can detect DDoS attacks and how accurate it is if DBSCAN is integrated with SVM in detecting DDoS attacks.

The structure of this research is described as follows. The second section describes related work. Implementation is the topic of discussion in Section III of this research. The fourth section discusses test results and analysis. In Section V, conclusions and future work are presented.

II. Related Works

A. DDoS Attack Detection with Machine Learning

DDoS attacks are undertaken in order to overwhelm targets and stop services from working. To initiate the attack, numerous devices are needed. Using a machine learning strategy is one technique to identify profitable and widely approved DDoS attacks [3]. In previous studies [6], DDoS attack detection has been carried out on an IoT-based banking sector monitoring system using SVM, RF, KNN which obtained an accuracy of 99.8%, 97.5%, and 98.74% respectively.

B. Hybrid Unsupervised and Supervised Model (Semi-supervised)

Semi-supervised learning is learning that uses clustering as well as classification. In this type of machine learning, unlabeled data are combined with additional information from labeled data [7]. In previous studies [8], DDoS attack detection has been carried out using a hybrid of K-Means and SVM which obtains a maximum accuracy of 99.9%.

C. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

One of the best unsupervised learning clustering methods is DBSCAN. Epsilon (ε) and $minPts$, the two parameters that this algorithm uses, must be pre-set in order to evaluate the point density distribution. The DBSCAN calculates the points that are within the radius of the surrounding area from the starting point smaller than the ε after starting from a place that has not been randomly visited. This method has the capacity to locate clusters and clean up noisy data. This technique can be used to differentiate between clusters of attack and regular data [5] [9] [10].

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:54 UTC from IEEE Xplore. Restrictions apply.

Assuming on the X dataset, the local density of item x_i is calculated using the DBSCAN algorithm, where $x_i \in X$ as the total number of items on $\varepsilon - neighboorhood$ (that is, cardinality of $N\varepsilon(x_i)$), where

$$N\varepsilon(x_i) = \{x_i \in X : \forall j, dist(x_i, x_j < \varepsilon) \quad (1)$$

On (1), DBSCAN has the ability to locate clusters of any shape. Furthermore, taking into amount of items in an environment, three types of items emerged. Core items are those whose "$\varepsilon - neighborhood$" contains at least $minPts$ of other items (that is, $|N\varepsilon(x_i)| \geq minPts$, for some $x_i \in X$). This implies, a cluster is formed against its neighboring items (together with the core item itself) for the given values of ε and $minPts$. However, for border items x_j, the core item x_i exists where $x_j \in N\varepsilon(x_i)$, even though the number of items in the $\varepsilon - neighborhood$ of border items is smaller than the specified $minPts$. Noisy items are the third category. These objects don't belong to any core item environments and their $\varepsilon - neighborhood$ doesn't contain at least a few $minPts$ items [11].

D. Support Vector Machine (SVM)

SVM is one method that can be utilized to enhance the anticipated outcome. This strategy is a kernel-based machine learning model that may be used for both regression and classification, according to Vapnik, who was the first to characterize it in that way. The machine learning, pattern recognition, and data mining sectors have recently shown a greater interest in SVM capabilities. SVM has been shown to be an effective method for resolving binary classification problems, therefore it can classify both normal classes and attack classes [12].

Suppose given a training dataset(x_1 , y_1 to x_n , y_n). Between the two data classes, a classifier with a wide margin will be found. A suitable set of $\vec{x}$ points can be used to describe each hyper-plane, where

$$\vec{w}.\vec{x} - b = 0 \quad (2)$$

The hyper-plane's vector normal is represented by $\vec{w}$. Hard margins and soft margins are the 2 types of margins. A hard margin will be utilized if the training data can be separated linearly and without any error (outliers and noise). To maximize the distance between the two data classes, linear SVM chooses two parallel hyper-planes that distinguish between them. The dataset in this case can be represented by two equations to label each class that is divided by a boundary by rescaling it, where

$$\vec{w}.\vec{x} - b = 1 \quad (3)$$

$$\vec{w}.\vec{x} - b = -1 \quad (4)$$

The classifier is set up, so that anything that is more than the threshold in (3) is assigned to one class, while anything that is less than the restrictions in (4) is assigned to a different class [13].

III. Implementation

A. Semi-supervised DBSCAN and SVM Model

The machine learning models for detecting DDoS attacks proposed in this study are semi-supervised DBSCAN and SVM. For more details can be seen in Fig. 1.

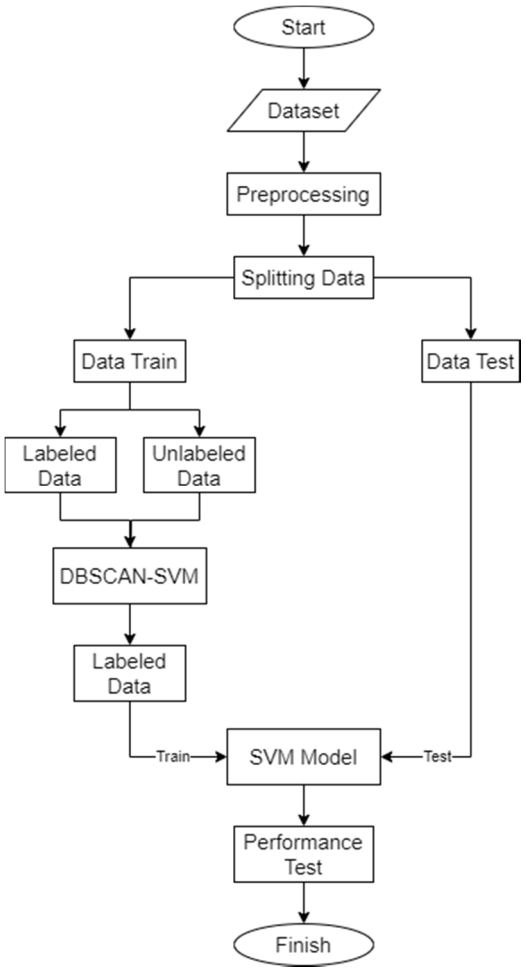


Fig. 1. Proposed DBSCAN and SVM model

First, the data goes through a preprocessing stage, this stage is explained in more detail in section III. Additionally, the data is split into train data and test data. The two sets of data were trained and tested using the semi-supervised DBSCAN and SVM models. Performance is tested using the confusion matrix generated from the model.

B. DDoS Attack Simulation

The DDoS attack simulation model proposed in this study is a DDoS attack simulation on the MQTT protocol as shown in Fig. 2. This simulation was carried out to obtain a simulation dataset.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:54 UTC from IEEE Xplore. Restrictions apply.

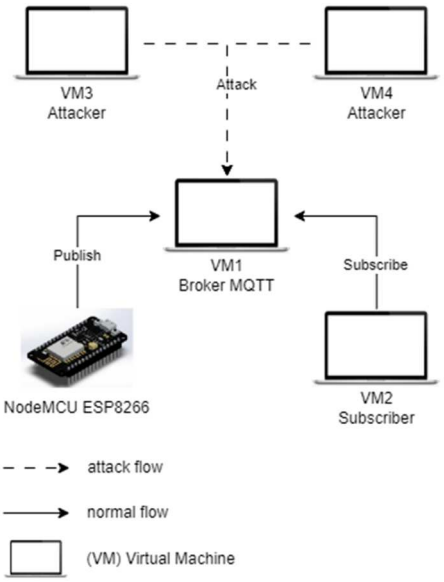


Fig. 2. Proposed DDoS attack simulation

The MQTT broker used is Mosquitto. There are 2 results captured, namely normal (benign) data and attack data. Normal data is obtained from VM1, VM2, and NodeMCU ESP8266 during publish, subscribe, MQTT broker connections, and other normal connections. Meanwhile, the attack data was obtained when VM3 and VM4 carried out a DDoS attack using the Hping3 software. The results of the data captured by Wireshark are then converted to csv using CICFlowMeter [14].

C. Datasets

In this research, three datasets are utilized so that the suggested model is more credible when applied to other datasets. The first dataset, IoTID20, was obtained from research [15] which has a total of 625783 data and 86 columns. However, not all data is used because there is data labeled as another attack. Therefore the data used from this dataset is only 80 columns, consisting of 183189 DDoS attack data and 40073 normal data. The second dataset comes from DDoS attack simulations performed on the MQTT protocol conducted in this research. This dataset has 100000 data and 80 columns containing 50000 DDoS attack data and 50000 normal data. The third dataset, CICDDOS2018, was obtained from research [16], the data used from this dataset is only 80000 data and 79 columns. This dataset consists of 40000 DDoS attack-HOIC data and 40000 normal data.

D. Preprocessing

There is some preprocessing carried out, such as converting categorical data to numeric, removing unusable features (object type), dealing with null and infinite values, value scaling (min-max scaler), and feature selection (Pearson's correlation coefficient with a 0.8 threshold). The feature selection gets different results.

On the IoTID20 dataset, the features resulting from feature selection are 38, as shown in Table I.

TABLE I. FEATURE SELECTION RESULTS ON IOTID20 DATASET

Feature Names	*Data Type*
Src_Port	Float64
Protocol	Float64
Flow_Duration	Float64
Tot_Fwd_Pkts	Float64
Tot_Bwd_Pkts	Float64
TotLen_Fwd_Pkts	Float64
TotLen_Bwd_Pkts	Float64
Fwd_Pkt_Len_Std	Float64
Bwd_Pkt_Len_Std	Float64
Flow_Byts/s	Float64
Flow_Pkts/s	Float64
Flow_IAT_Mean	Float64
Flow_IAT_Std	Float64
Fwd_IAT_Tot	Float64
Fwd_IAT_Std	Float64
Bwd_IAT_Std	Float64
Fwd_PSH_Flags	Float64
Bwd_PSH_Flags	Float64
Fwd_URG_Flags	Float64
Bwd_URG_Flags	Float64
Bwd_Pkts/s	Float64
Pkt_Len_Std	Float64
FIN_Flag_Cnt	Float64
SYN_Flag_Cnt	Float64
RST_Flag_Cnt	Float64
CWE_Flag_Count	Float64
Down/Up_Ratio	Float64
Fwd_Byts/b_Avg	Float64
Fwd_Pkts/b_Avg	Float64
Fwd_Blk_Rate_Avg	Float64
Bwd_Byts/b_Avg	Float64
Bwd_Pkts/b_Avg	Float64
Bwd_Blk_Rate_Avg	Float64
Init_Fwd_Win_Byts	Float64
Init_Bwd_Win_Byts	Float64
Fwd_Seg_Size_Min	Float64
Active_Mean	Float64
Active_Std	Float64

An example of data from the IoTID20 dataset is shown in Table II.

TABLE II. SAMPLE DATA ON IOTID20 DATASET

Src_Port	*Protocol*	*Flow_Duration*	*…*	*Label*
0.006838	0.352941	0.154827	…	1
0.928716	1.000000	0.051002	…	1
0.866153	1.000000	0.502732	…	1
0.139234	0.352941	0.435337	…	0
0.154361	1.000000	0.358834	…	1

As indicated in Table III, the simulation dataset's 37 features are the result of feature selection.

TABLE III. FEATURE SELECTION RESULTS ON SIMULATION DATASET

Feature Names	*Data Type*
Src Port	Float64
Dst Port	Float64
Protocol	Float64
Total Fwd Packet	Float64
Total Bwd packets	Float64
Total Length of Bwd Packet	Float64
Fwd Packet Length Std	Float64
Bwd Packet Length Max	Float64
Bwd Packet Length Min	Float64
Flow Bytes/s	Float64
Flow Packets/s	Float64
Flow IAT Std	Float64
Bwd IAT Total	Float64
Bwd IAT Min	Float64
Bwd PSH Flags	Float64
Fwd URG Flags	Float64
Bwd URG Flags	Float64
Fwd Header Length	Float64

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:54 UTC from IEEE Xplore. Restrictions apply.

Feature Names	*Data Type*
Bwd Packets/s	Float64
Packet Length Std	Float64
Packet Length Variance	Float64
FIN Flag Count	Float64
SYN Flag Count	Float64
RST Flag Count	Float64
URG Flag Count	Float64
CWR Flag Count	Float64
ECE Flag Count	Float64
Fwd Bytes/Bulk Avg	Float64
Fwd Packet/Bulk Avg	Float64
Fwd Bulk Rate Avg	Float64
Bwd Packet/Bulk Avg	Float64
Bwd Bulk Rate Avg	Float64
Subflow Bwd Packets	Float64
Subflow Bwd Bytes	Float64
FWD Init Win Bytes	Float64
Bwd Init Win Bytes	Float64
Active Mean	Float64

Table IV displays a sample of data from the simulation dataset.

TABLE IV. SAMPLE DATA ON SIMULATION DATASET

Src Port	*Dst Port*	*Protocol*	*…*	*Label*
0.532494	0.031279	1.000000	…	1
0.028733	0.750133	0.352941	…	0
0.838407	0.031279	1.000000	…	1
0.755459	0.031279	1.000000	…	1
0.719249	0.031279	1.000000	…	1

The 27 features from the CICDDOS2018 dataset as a consequence of feature selection are shown in Table V.

TABLE V. FEATURE SELECTION RESULTS ON CICDDOS2018 DATASET

Feature Names	*Data Type*
Dst Port	Float64
Protocol	Float64
Tot Bwd Pkts	Float64
Fwd Pkt Len Min	Float64
Bwd Pkt Len Min	Float64
Flow Byts/s	Float64
Flow Pkts/s	Float64
Flow IAT Mean	Float64
Bwd IAT Tot	Float64
Bwd IAT Mean	Float64
Bwd IAT Min	Float64
Fwd PSH Flags	Float64
Bwd PSH Flags	Float64
Fwd URG Flags	Float64
Bwd URG Flags	Float64
Bwd Pkts/s	Float64
FIN Flag Cnt	Float64
URG Flag Cnt	Float64
CWE Flag Count	Float64
Fwd Byts/b Avg	Float64
Fwd Pkts/b Avg	Float64
Fwd Blk Rate Avg	Float64
Bwd Byts/b Avg	Float64
Bwd Pkts/b Avg	Float64
Bwd Blk Rate Avg	Float64
Fwd Act Data Pkts	Float64
Active Std	Float64

A sample of the data from the CICDDOS2018 dataset is shown in Table VI.

TABLE VI. SAMPLE DATA ON CICDDOS2018 DATASET

Dst Port	*Protocol*	*Tot Bwd Pkts*	*…*	*Label*
0.001221	0.352941	0.000000	…	1
0.965819	0.352941	0.016129	…	0
0.001221	0.352941	0.000000	…	1
0.001221	0.352941	0.032258	…	1
0.968719	0.352941	0.016129	…	0

E. Test Scenario

From the preprocessed dataset, we reduce the dimensional to only two dimensional using Principal Component Analysis (PCA) to reduce the clustering process time by DBSCAN. The data is split into train data and test data. In the training data, there are 50% unlabeled data and 50% labeled data. DBSCAN is used to process training and test data through clustering. After carrying out the clustering process, the initial SVM is trained using labeled data that already has a cluster label from the clustering results. Unlabeled data is predicted using the initial SVM. After all the training data has been labeled, the latest SVM model is trained using the training data. Furthermore, the test data will be tested using the latest SVM model. This study uses a ratio of 70:30 for each training data and test data.

F. Evaluation Metrics

The evaluation metric used to test the model in this study is a metric that has been widely used in previous studies, namely accuracy (5), f1-score (8), and false alarm rate (9) from the resulting confusion matrix [17] [18]. The formula is as follows.

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN} * 100 \quad (5)$$

$$precision = \frac{TP}{TP + FP} * 100 \quad (6)$$

$$recall = \frac{TP}{TP + FN} * 100 \quad (7)$$

$$f1 - score = \frac{2 * (precision * recall)}{precision + recall} * 100 \quad (8)$$

$$false\ alarm\ rate = \frac{FP}{FP + TN} * 100 \quad (9)$$

The above results are obtained from the confusion matrix table as shown in Table VII. The number of attack data labeled by the model as attack data is known as True Positive (TP). The number of normal data that the model has labeled as normal data is known as True Negative (TN). The number of normal data that the model incorrectly labels as attack data is known as False Positive (FP). The number of attack data that the model labels as normal data is known as a false negative (FN).

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:54 UTC from IEEE Xplore. Restrictions apply.

TABLE VII. CONFUSION MATRIX TABLE

		Predicted Values	
		0	1
Actual Values	0	TN	FP
	1	FN	TP

IV. EVALUATION AND ANALYSIS

A. Test Results

We divide the test results into three parts, namely when using the IoTID20 dataset, simulation dataset, and CICDDOS2018 dataset.

It can be seen in Table VIII, when using the IoTID20 dataset in our proposed model, attack data and normal data can be detected quite well which results in accuracy and f1-score are 91% and 94.5% respectively, but the false alarm rate is still quite high at 24.4%.

TABLE VIII. TEST RESULTS OF IOTID20 DATASET

	Values
Accuracy	91%
Precision	94.7%
Recall	94.3%
F1-score	94.5%
False Alarm Rate	24.4%

These results are obtained from the confusion matrix table in Fig. 3.

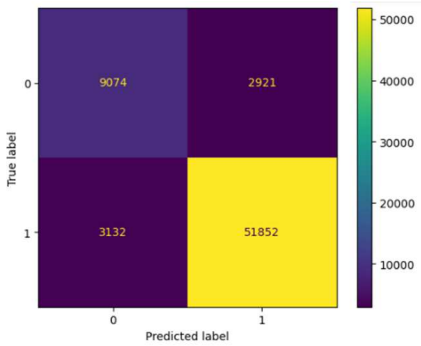


Fig. 3. Confusion matrix of IoTID20 dataset

As shown in Table IX, our proposed model can detect attack data and normal data very well when utilizing simulation datasets, resulting in accuracy, f1-score, and false alarm rate of 99.4%, 99.4%, and 1.2% respectively.

TABLE IX. TEST RESULTS OF SIMULATION DATASET

	Values
Accuracy	99.4%
Precision	98.8%
Recall	100%
F1-score	99.4%
False Alarm Rate	1.2%

The confusion matrix table in Fig. 4 provided these results.

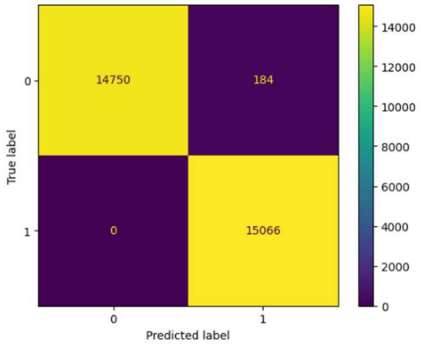


Fig. 4. Confusion matrix of simulation dataset

As shown in Table X, our proposed model performs well when applied to the CICDDOS2018 dataset and is able to recognize attack and normal data with best accuracy, f1-score, and false alarm rate of 99.6%, 99.6%, and 0.8% respectively.

TABLE X. TEST RESULTS OF CICDDOS2018 DATASET

	Values
Accuracy	99.6%
Precision	99.2%
Recall	100%
F1-score	99.6%
False Alarm Rate	0.8%

These results can be found in the confusion matrix table in Fig. 5.

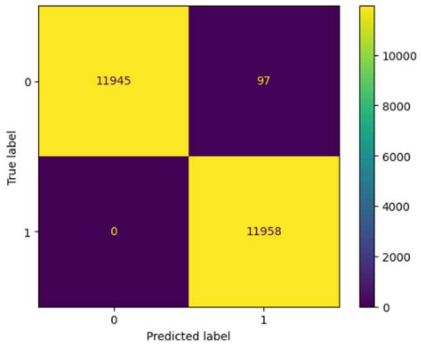


Fig. 5. Confusion matrix of CICDDOS2018 dataset

B. Analysis of Test Results

As can be seen in Table XI, we compared our proposed model for each dataset.

TABLE XI. TEST RESULTS COMPARISON OF EACH DATASET

Dataset	***Accuracy***	***F1-score***	***False Alarm Rate***
IoTID20 Dataset	91%	94.5%	24.4%
Simulation Dataset	99.4%	99.4%	1.2%
CICDDOS2018 Dataset	**99.6%**	**99.6%**	**0.8%**

We provide a comparison of test results in the form of graphical visualizations, as shown in Fig. 6 to make it easier to read the comparison of test results for each dataset. Unlike accuracy and f1-score, the false alarm rate result is better when the false alarm rate is lower.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:54 UTC from IEEE Xplore. Restrictions apply.

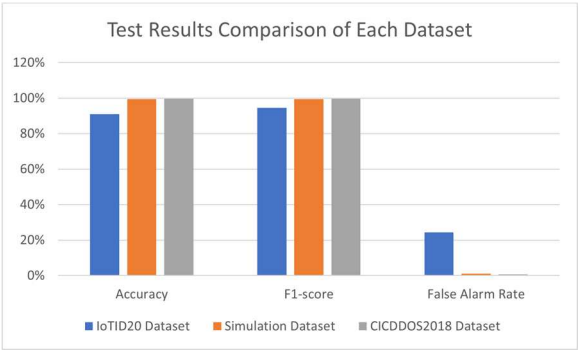


Fig. 6 Test results comparison of each dataset on graph representation

From the results of the tests carried out, the model we proposed obtained different results, but both obtained fairly good results. The accuracy obtained is in the range of 91% to 99.6%. The F1-score obtained is in the range of 94.5% to 99.6%. The false alarm rate obtained when using the IoTID20 dataset is still quite high compared to the simulation dataset and CICDDOS2018 dataset. This is caused by imbalanced data in the IoTID20 dataset, which has more attack data than normal data (183189 vs. 40073). The simulation dataset and CICDDOS2018 dataset which have balanced data get very high accuracy and f1-score and also a very low false alarm rate.

V. Conclusions

Based on the findings of this study's experiments, semi-supervised DBSCAN and SVM models can detect DDoS attacks quite effectively. For each dataset, the performance of the machine learning models utilized in this study generated varied accuracy, f1-scores, and false alarm rates. This is a consequence of the ε and $minPts$ parameter's setting values being different in the DBSCAN model. The unbalanced dataset, which included both attack and benign data, was another contributing factor. The maximum accuracy and f1 score, at 99.6% and 99.6%, respectively, and the lowest false alarm rate, at 0.8%, were achieved by using balanced data and optimal DBSCAN parameters.

Our recommendations for the future work include creating a dataset with balanced data from each class to lower the false alarm rate. Another idea is to use a different dimensional reduction technique to improve DBSCAN's processing time and clustering results.

References

[1] S. M. Tahsien, H. Karimipour, and P. Spachos, "Machine learning based solutions for security of Internet of Things (IoT): A survey," *Journal of Network and Computer Applications*, vol. 161, Jul. 2020, doi: 10.1016/j.jnca.2020.102630.

[2] F. Chen, Y. Huo, J. Zhu, and D. Fan, "A Review on the Study on MQTT Security Challenge," in *IEEE International Conference on Smart Cloud*, Institute of Electrical and Electronics Engineers Inc., Nov. 2020, pp. 128–133. doi: 10.1109/SmartCloud49737.2020.00032.

[3] N. Mishra and S. Pandya, "Internet of Things Applications, Security Challenges, Attacks, Intrusion Detection, and Future Visions: A Systematic Review," *IEEE Access*, vol. 9, pp. 59353–59377, 2021, doi: 10.1109/ACCESS.2021.3073408.

[4] V. Deepa, K. Muthamil Sudar, and P. Deepalakshmi, "Detection of DDoS Attack on SDN Control plane using Hybrid Machine Learning Techniques," in *International Conference on Smart Systems and Inventive Technology*, IEEE, 2018, pp. 299–303.

[5] M. Somina, "Web Vulnerability DDOS attack detection in cloud using DBSCAN clustering," *International Journal of Advanced Studies of Scientific Research*, vol. 4, no. 1, 2018, [Online]. Available: https://ssrn.com/abstract=3329414.

[6] U. Islam *et al.*, "Detection of Distributed Denial of Service (DDoS) Attacks in IOT Based Monitoring System of Banking Sector Using Machine Learning Models," *Sustainability (Switzerland)*, vol. 14, no. 14, Jul. 2022, doi: 10.3390/su14148374.

[7] S. Soheily-Khah, P. F. Marteau, and N. Bechet, "Intrusion detection in network systems through hybrid supervised and unsupervised machine learning process: A case study on the iscx dataset," in *International Conference on Data Intelligence and Security*, Institute of Electrical and Electronics Engineers Inc., May 2018, pp. 219–226. doi: 10.1109/ICDIS.2018.00043.

[8] R. Jaiswar, "DDoS Attack prediction and classification at Application Layer for Web protocol using Kmeans-SVM Machine Learning Algorithm," National College of Ireland, 2021.

[9] D. Luchi, A. Loureiros Rodrigues, and F. Miguel Varejão, "Sampling approaches for applying DBSCAN to large datasets," *Pattern Recognit Lett*, vol. 117, pp. 90–96, Jan. 2019, doi: 10.1016/j.patrec.2018.12.010.

[10] Y. Yang *et al.*, "An efficient DBSCAN optimized by arithmetic optimization algorithm with opposition-based learning," *J Supercomput*, vol. 78, no. 18, pp. 19566–19604, Dec. 2022, doi: 10.1007/s11227-022-04634-w.

[11] A. A. Bushra and G. Yi, "Comparative Analysis Review of Pioneering DBSCAN and Successive Density-Based Clustering Algorithms," *IEEE Access*, vol. 9, pp. 87918–87935, 2021, doi: 10.1109/ACCESS.2021.3089036.

[12] J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez, "A comprehensive survey on support vector machine classification: Applications, challenges and trends," *Neurocomputing*, vol. 408, pp. 189–215, Sep. 2020, doi: 10.1016/j.neucom.2019.10.118.

[13] S. Ghosh, A. Dasgupta, and A. Swetapadma, "A Study on Support Vector Machine based Linear and Non-Linear Pattern Classification," in *International Conference on Intelligent Sustainable Systems*, 2019, pp. 24–28.

[14] A. Habibi Lashkari, "CICFlowMeter v4.0." 2017. Accessed: May 26, 2023. [Online]. Available: https://github.com/ahlashkari/CICFlowMeter.

[15] I. Ullah and Q. H. Mahmoud, "A Scheme for Generating a Dataset for Anomalous Activity Detection in IoT Networks," in *Canadian Conference on Artificial Intelligence*, Springer, 2020, pp. 508–520. doi: 10.1007/978-3-030-47358-7_52.

[16] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *International Conference on Information Systems Security and Privacy*, SciTePress, 2018, pp. 108–116. doi: 10.5220/0006639801080116.

[17] T. A. Tuan, H. V. Long, L. H. Son, R. Kumar, I. Priyadarshini, and N. T. K. Son, "Performance evaluation of Botnet DDoS attack detection using machine learning," *Evol Intell*, vol. 13, no. 2, pp. 283–294, Jun. 2020, doi: 10.1007/s12065-019-00310-w.

[18] F. S. De Lima Filho, F. A. F. Silveira, A. De Medeiros Brito Junior, G. Vargas-Solar, and L. F. Silveira, "Smart Detection: An Online Approach for DoS/DDoS Attack Detection Using Machine Learning," *Security and Communication Networks*, vol. 2019, 2019, doi: 10.1155/2019/1574749.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:54 UTC from IEEE Xplore. Restrictions apply.